SAFI INSTITUTE OF ADVANCED STUDY (AUTONOMOUS)

Affiliated to University of Calicut, Accredited by NAAC with A++ Grade(3.54)

Recognized by UGC under 2(f),ISO 9001-2015 Certified by AAB

Vazhayur East, Malappuram Dist, Kerala– 673633

Department of Computer Applications

Project Report

EduGraph

Done by,

1. **AJUL LAL P - (SIAWBCA041)**
2. **AJMAL SHAREEF MP - (SIAWBCA006)**
3. **ADITH V - (SIAWBCA020)**
4. **MUHAMMED BISHRE KT - (SIAWBCA047)**

Under the Guidance of

Mr. Anshid Babu TM

2022-2025

SAFI INSTITUTE OF ADVANCED STUDY (AUTONOMOUS)

Affiliated to University of Calicut, Accredited by NAAC with A++ Grade (3.54)

Recognized by UGC under 2(f), ISO 9001-2015 Certified by AAB

Vazhayur East, Malappuram Dist, Kerala– 673633

Department of Computer Applications

CERTIFICATE

Certified that this is the bonafide report on the project work entitled

EduGraph

Done by,

1. **AJUL LAL P - (SIAWBCA041)**
2. **AJMAL SHAREEF MP - (SIAWBCA006)**
3. **ADITH V - (SIAWBCA020)**
4. **MUHAMMED BISHRE KT - (SIAWBCA047)**

During the academic year 2022-2025 in partial fulfilment of requirement for the award of the degree of,

Bachelor of Computer Application

Of

University of Calicut, Malappuram Dist., Kerala

Internal Guide **Head of the Department**

Internal Examiner **External Examiner**

SAFI INSTITUTE OF ADVANCED STUDY
Rasiya Nagar, Vazhayur East P.O., Ramanattukara Via,
Malappuram District - 673 633, Kerala.
Ph: 0483 - 2880000 (100 Lines)
www.sias.edu.in mail@sias.edu.in

Affiliated to the University of Calicut.
Recognised by UGC 2 (f)
Accredited by NAAC with **'A++ Grade'** (3.54)

Date : 18 March 2025

Place : Vazhayur

TO WHOM IT MAY CONCERN

This is to certify that **Mr. AJUL LAL P (REG.NO: SIAWBCA041), AJMAL SHAREEF MP (REG.NO: SIAWBCA006), ADITH V (REG.NO: SIAWBCA020), MUHAMMED BISHRE KT(REG.NO: SIAWBCA047)**, a 6th semester student pursuing **Bachelor of Computer Application (BCA) at SAFI Institute of Advanced Study (Autonomous), Vazhayur, Malappuram** has successfully completed their academic project on **RESULT ANALYSIS SYSTEM** using **PYTHON DJANGO** from December 2024 to March 2025. Throughout the duration of their academic project, they demonstrated punctuality, diligence, and curiosity. We wish them every success in their future endeavors.

Principal

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for His blessings, guidance, and grace, which strengthened and guided us throughout our academic journey.

We are immensely grateful to the Principal, Prof. E. P. Imbichi Koya for their visionary leadership, unwavering support, and commitment to fostering an environment of excellence and innovation within our institute.

Our sincere thanks go to the Head of the Department, Mr. Abdul Samad C. for their invaluable guidance and encouragement, which have been instrumental in nurturing our intellectual curiosity and facilitating our academic growth.

We extend our heartfelt appreciation to our Project Guide Mr.Anshid Babu TM for his/her wisdom and guidance have been invaluable assets in our academic journey.

We would also like to express our gratitude to our teachers for their commitment has been instrumental in shaping our understanding and appreciation of the field of Computer Science.

Lastly, we sincerely thank our friends for their companionship, support, and encouragement, which made this journey memorable. Together, we achieved great milestones and move forward with knowledge, skills, and lasting memories. Thank you all for your unwavering support.

Ajul Lal P

Ajmal Shareef MP

Adith V

Muhammed Bishre KT

BCA 2022-25

Department of Computer Applications

SAFI Institute of Advanced Study(Autonomous), Vazhayur

Table of Contents

1. ABSTRACT……1

2. INDRODUCTION……..3

2.1 Overview of system……4

2.2 Software configuration……...5

2.3 Hardware configuration……..6

3. SYSTEM ANALYSIS……7

3.1 Existing system……...8

3.2 Proposed system……8

3.3 Feasibility study…… 8

3.4 Project planning……11

3.5 Data Flow Diagram…….. 13

3.6 Software requirement specification……..13

4. SYSTEM DESIGN……16

4.1 Input & output design……...17

4.2 Process design…….. 18

5. TESTING & IMPLEMENTATION……...20

6. CONCLUSION……23

7. SCOPE FOR FUTURE DEVELOPMENT……25

8. APPENDIX……27

8.1 Source code…….. 28

8.2 GUI Screenshots……...54

8.3 System minimum requirements……58

9. REFERENCES…… 59

9.1 Book references……60

9.2 Website references……... 60

1. ABSTRACT

EDUGRAPH - RESULT ANALYSIS SYSTEM

The EduGraph is a result analysis system is designed to automatically analyse the results of students in a class by processing individual result PDFs. The current system only displays individual results on separate pages without providing an overall view of all students' performance or graphical analysis. This makes it challenging for teachers to analyse the results of all students collectively.

The proposed system addresses these issues by consolidating the results of all students on a single page and providing graphical analysis of performance. It collects all individual result PDFs, processes them, and converts the analysed data into a downloadable format. This system will improve efficiency by saving time and offering a better understanding of class performance through visual representations.

The primary technologies used to develop the system are Python, the Django framework, pdfplumber, and xhtml2pdf.

Project Members: Ajul Lal P
Ajmal Shareef M.P
Adith V
Muhammed Bishre K.T

Project Guide: Anshid Babu T.M

2. INTRODUCTION

INTRODUCTION

2.1 Overview of the system

The "EduGraph" is a digital platform designed to enhance the assessment process within educational institutions by providing streamlined methods for analysing student performance. Through this system, teachers can access a centralized hub to efficiently process and evaluate students' results. The platform collects individual result PDFs, analyses them, and generates consolidated reports in a downloadable format.

Leveraging modern technologies such as Python, the Django framework, pdfplumber, and xhtml2pdf, the system ensures accuracy and efficiency in performance analysis. By offering graphical representations and collective insights, the EduGraph aims to improve the understanding of class performance, promote data-driven decision-making, and enhance the overall academic evaluation process.

SYSTEM ENVIRONMENT

System environment is the term commonly used to refer to support an application. A software environment for a particular application could include the operating system, the data base system, specific development tool or compiler.

2.2 Software configuration

For the proposed EduGraph result analysis system to function efficiently, the following software components must be installed and running on the server/client:

- Operating System: Windows 7 or above / Linux (latest versions)
- Programming Languages: Python, JavaScript
- Integrated Development Environment (IDE): Visual Studio Code
- Frontend Technologies: HTML, JavaScript, CSS
- Backend Technologies: Django
- Other Tools: pdfplumber, xhtml2pdf

2.2.1 Python

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its simple syntax emphasizes readability, reducing program maintenance costs. Python supports modules and packages, promoting modularity and code reuse. EduGraph uses Python for backend processing and result extraction.

2.2.2 JavaScript

JavaScript is a dynamic, multi-paradigm language used for client-side scripting. Its syntax is derived from Java and C, making it widely adopted for interactive web applications. EduGraph utilizes JavaScript for frontend interactivity and dynamic content rendering.

2.2.3 HTML

HTML (HyperText Markup Language) is a markup language used to create web pages and applications. EduGraph employs HTML for structuring the user interface and result display pages.

2.2.4 CSS

CSS (Cascading Style Sheets) is used to control the visual presentation of HTML documents. EduGraph uses CSS to enhance layout styling, responsiveness, and overall UI design.

2.2.5 Django

Django is a high-level Python web framework that facilitates rapid development and secure web applications. EduGraph utilizes Django for:

- Managing result processing workflows
- Generating reports and serving structured academic data

2.3 Hardware Configuration

For optimal performance, the following minimum hardware specifications are recommended:

- **Processor:** Intel Pentium IV or above
- **Monitor:** Minimum 14 inches
- **RAM:** 4 GB
- **Hard Disk:** 80 GB or more
- **Mouse:** Serial mouse

3. SYSTEM ANALYSIS

SYSTEM ANALYSIS

The system analysis for the "EduGraph" result analysis system project involves thoroughly understanding the needs of educational institutions in evaluating student performance. The current process requires manual checking of individual results, making it time-consuming and inefficient for teachers to assess overall class performance.

The proposed system streamlines this process by automatically analysing student results from individual PDFs and consolidating them into a structured, downloadable format. This ensures better data management, reduces manual workload, and provides graphical insights into student performance trends. By leveraging technologies such as Python, the Django framework, pdfplumber, and xhtml2pdf, the system enhances efficiency, accuracy, and accessibility in academic evaluation.

3.1 Existing System

The current system for analysing student results requires manually reviewing individual result PDFs, making it challenging to gain an overall understanding of class performance. Teachers must navigate multiple documents separately, which is both time-consuming and inefficient. Additionally, there is no provision for graphical representation or consolidated reporting, limiting insights into student performance trends.

3.2 Proposed System

The proposed system, EduGraph, is a web-based result analysis system designed to automate and enhance academic performance evaluation. The system extracts, processes, and visualizes student results, providing structured performance reports through an intuitive interface. EduGraph eliminates the challenges of manual result processing by offering an automated, efficient, and data-driven solution.

3.3 Feasibility Study

A feasibility study was conducted to evaluate the practicality of implementing the EduGraph system based on technical, operational, economic, and scheduling aspects.

3.3.1. Technical Feasibility:

Assessment:

System Requirements: Determines whether the system can be developed using existing technologies.

Integration: Evaluates compatibility with institutional databases and authentication systems.

Scalability: Assesses whether the system can accommodate increasing student data over time.

Findings:

The system is developed using Django for the backend and HTML, JavaScript, and SCSS for the frontend.

Integration with existing institutional databases may require API development or structured data migration.

The system is designed with scalability in mind, ensuring efficient performance even with a growing user base.

3.3.2. Operational Feasibility:

Assessment:

User Acceptance: Evaluates the adaptability of faculty and students to the new system.

Training Needs: Determines whether users require training to operate the system effectively.

Maintenance: Assesses the feasibility of long-term system support and updates.

Findings:

User participation during development will enhance adoption and usability.

The system provides an intuitive user interface, requiring minimal training for faculty and students.

System maintenance can be managed internally by the institution's IT team or outsourced as needed.

3.3.3. Economic Feasibility:

Assessment:

Cost Analysis: Estimates the initial investment and ongoing operational expenses.

Return on Investment (ROI): Determines whether the benefits outweigh the costs.

Cost-Benefit Analysis: Compares the financial implications of implementing the system against alternative solutions.

Findings:

Initial development costs include software implementation, hosting, and user training.

Recurring costs involve maintenance, software updates, and hosting expenses.

The system significantly reduces manual workload and administrative errors, resulting in a high ROI through improved efficiency.

3.3.4. Scheduling Feasibility:

Assessment:

Project Timeline: Determines the feasibility of completing the project within the allocated timeframe.

Resource Availability: Evaluates access to skilled personnel and necessary technological resources.

Risk Management: Identifies challenges and mitigation strategies to prevent implementation delays.

Findings:

A structured development plan with clearly defined milestones ensures timely completion.

The project is supported by experienced developers and domain experts.

Risk assessment and contingency planning are incorporated to minimize disruptions and ensure smooth deployment.

3.4 Project planning

The project planning for the EduGraph result analysis system involves several critical steps to ensure the successful implementation of the system within the specified timeframe and budget constraints. The planning process is structured to enhance academic result management, automate performance evaluation, and improve data-driven decision-making.

The first step in project planning is to define the project objectives. These objectives should align with institutional goals by automating result processing, enhancing data analysis capabilities, and providing structured performance reports. The system aims to reduce manual workload, improve report accuracy, and offer interactive result visualization.

Once the objectives are established, the project scope needs to be clearly outlined. This includes identifying the functional requirements such as:

- Result processing to extract and analyse student performance data
- Graphical representation of results through interactive visualizations
- Automated report generation for structured performance evaluation

Additionally, non-functional requirements such as system performance, security, and usability standards must be defined to ensure system reliability and user accessibility. It is crucial to establish what features will be included in the system and what aspects will be reserved for future upgrades.

Developing a project timeline is essential to ensure systematic project execution. The timeline should include key milestones and deliverables such as:

1. Project Initiation – Requirement gathering and feasibility study
2. System Design – Architectural planning and module design
3. Development Phase – Coding and system implementation
4. Testing Phase – Functional testing, bug fixes, and performance validation
5. Deployment – System rollout and live implementation
6. Post-Implementation Support – Continuous monitoring and system updates

Each phase will have clearly defined objectives and deadlines to ensure project efficiency and timely completion.

Resource allocation is another critical aspect of project planning. This includes assigning the necessary resources required for successful implementation, such as:

- Human resources – Software developers, project managers, testers, and system administrators
- Technical infrastructure – Hardware, software tools, and cloud services
- Financial resources – Budget allocation for development, hosting, licensing, training, and system maintenance

Risk management is essential to anticipate potential challenges and develop mitigation strategies. Risks could include technical constraints, data integration challenges, or project timeline risks. Risk management measures will be in place to minimize disruptions and ensure smooth execution.

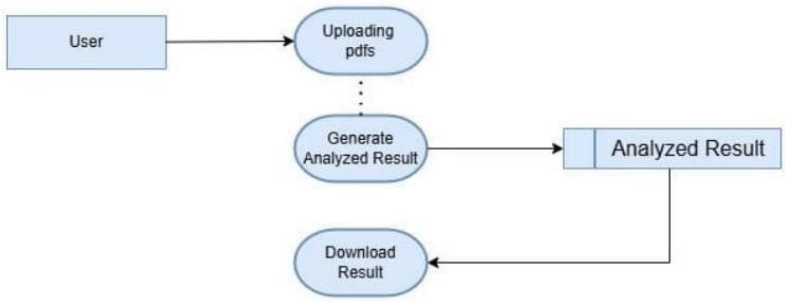
Finally, quality assurance processes will be implemented to maintain system integrity and compliance with project requirements. This includes:

- Testing strategies such as unit testing, integration testing, and user acceptance testing
- Regular code reviews and inspections to identify and resolve errors early
- Continuous improvement strategies to optimize system performance and enhance user experience

3.5 Data Flow Diagram

Level 0

Level 1



3.6 Software Requirement Specification

1. Purpose:

The purpose of the EduGraph result analysis system is to provide an automated and centralized platform for processing and analysing student academic results. The system aims to streamline result management, enhance data visualization, and improve academic performance evaluation. EduGraph eliminates the inefficiencies of manual result processing by offering an efficient and structured approach to student performance analysis and report generation.

2. Scope:

User Management: Authentication and authorization of users, including Administrators, Faculty, and Students.

Result Processing: Extraction and analysis of student results from uploaded PDF files.

Performance Analysis: Generation of pass/fail counts, subject-wise failure statistics, and student rankings.

Report Generation: Automated creation of structured academic reports in PDF format for institutional review and record-keeping.

Data Visualization: Graphical representation of student performance trends for enhanced analysis.

3. Functional Requirements:

Result Processing:

- Upload student result PDFs for automatic processing.
- Extract student details and grades from the uploaded files.
- Calculate pass/fail counts and generate subject-wise failure statistics.
- Identify top-performing students based on SGPA.

Report Generation:

- Generate detailed student performance reports in PDF format.
- Provide structured analysis of academic trends and ranking data.
- Enable users to download reports for reference and record-keeping.

Data Visualization:

- Display interactive graphical representations of student performance trends.
- Provide subject-wise and overall performance insights through visual analytics.

Request Handling:

- Allow users to upload and process result files.
- Authenticate and verify uploaded result data for accuracy.

Notification System:

- Send real-time notifications regarding the status of result processing.

- Notify users when reports are available for download.

4. Non-Functional Requirements:

Performance:

- The system should be responsive and able to handle multiple result processing requests simultaneously.
- Response time for processing and generating reports should be within acceptable limits.

Security:

- Data protection mechanisms should be implemented to prevent unauthorized modifications.
- Encryption techniques should be used to safeguard sensitive academic data.

Usability:

- The user interface should be intuitive and easy to navigate.
- The system should provide a seamless experience for uploading, processing, and downloading results.

Reliability:

- The system should ensure consistent availability for result processing.
- Backup and recovery mechanisms should be in place to prevent data loss.

Scalability:

- The system should be designed to accommodate an increasing number of result files and users.
- Performance should remain stable as the volume of academic data grows.

4. SYSTEM DESIGN

SYSTEM DESIGN

4.1 Input & Output Design

Module	Functionality	Input	Output
Upload Module	Allows users to upload multiple student result PDFs. Ensures proper file handling and storage.	Student result PDFs	Extracted student details and grades
Result Processing Module	Reads and extracts student results. Computes pass/fail counts, ranks students, and identifies subject-wise failures.	Extracted data from PDFs	Structured result data, student rankings
Report Generation Module	Converts processed review.	Structured result data.	Downloadable PDF report.
Data Visualisation Module	Displays interactive Analysis.	Processed students data.	Graphical performance insights.
Downloading Module	Enables users to download analysed results.	Processed students performance data.	Downloadable academic reports in PDF format.

4.2 Process design

The process design for the EduGraph result analysis system involves mapping out the workflow and activities related to result extraction, processing, analysis, and report generation. The process begins with the uploading of student result PDFs, which are then processed to extract relevant academic data. The system automatically computes pass/fail counts, subject-wise failures, and student rankings based on SGPA.

Once the data is processed, structured reports are generated in PDF format, allowing users to review detailed performance insights. The system also provides interactive graphical representations of student results for enhanced analysis. Users can then download the reports for reference and institutional record-keeping.

Throughout the process, accuracy and efficiency are prioritized to ensure seamless result analysis. The system is designed for automated data extraction and processing, minimizing manual intervention and improving the overall academic evaluation process. Continuous monitoring mechanisms are implemented to optimize performance and address any potential issues.

4.2.1 Upload Module

The Upload Module allows users to upload multiple student result PDFs for processing. This module ensures proper file handling and storage while maintaining data integrity. Once processing is complete, the uploaded files are automatically deleted to optimize system efficiency.

4.2.2 Result Processing Module

The Result Processing Module extracts and processes student result data from uploaded PDFs. It performs the following functions:

- Reads and extracts student details and grades from the uploaded documents.
- Computes pass/fail counts for academic performance evaluation.
- Identifies the top three students based on SGPA while ensuring rank uniqueness.
- Determines subject-wise failure counts to analyse academic weaknesses.
- Generates structured output, allowing faculty and administrators to review and analyse results effectively.

4.2.3 Download Module

The Download Module enables users to download the final result analysis as a PDF report. This ensures that the processed results are formatted for reference and record-keeping. The module facilitates quick access to structured academic reports for institutional use.

5. TESTING AND IMPLEMENTATION

TESTING AND IMPLEMENTATION

To ensure the successful testing and implementation of the EduGraph result analysis system, a structured approach is necessary. The system development follows a well-defined plan, beginning with backend processing and result extraction. The implementation focuses on automating student result analysis, report generation, and data visualization.

The backend development is carried out using Django, where result extraction logic and report generation functionalities are implemented. The system processes student result PDFs using tools like pdfplumber for extraction and xhtml2pdf for structured report generation. API endpoints are defined for handling file uploads, processing requests, and generating reports, ensuring seamless functionality.

Simultaneously, frontend development involves designing an intuitive user interface using HTML, CSS, and JavaScript. The UI facilitates file uploads, graphical result visualization, and report downloads, ensuring a seamless user experience.

Testing is performed at multiple levels to validate system functionality:

- Unit Testing: Each module, including file processing, result extraction, and PDF report generation, is tested independently using appropriate testing frameworks.
- Integration Testing: Interactions between modules are tested to ensure smooth data processing, graphical representation, and seamless PDF report generation. Scenarios such as file uploads, result analysis, and report downloads are thoroughly validated.
- User Acceptance Testing (UAT): The system is tested by end-users to ensure that it meets expectations in terms of usability, accuracy, and efficiency. Feedback is incorporated to refine performance and improve functionality.

Finally, the system is deployed, followed by continuous monitoring and maintenance to ensure reliable performance. The EduGraph result analysis system undergoes periodic updates to enhance efficiency, accuracy, and report generation capabilities.

5.1 Testing Techniques

Unit Testing:

- Write unit tests for each module's functionalities to ensure they work as expected.
- Test result extraction, PDF processing, report generation, and graphical representation.
- Use testing frameworks like pytest for backend logic validation.

Integration Testing:

- Test interactions between different modules to ensure seamless data processing.
- Validate scenarios such as file uploads, result analysis, and PDF report generation.
- Ensure that extracted student data is correctly formatted and displayed.

User Acceptance Testing (UAT):

- Involve end-users to validate the system against their requirements.
- Conduct UAT sessions to ensure that the system meets expectations in terms of usability, accuracy, and performance.
- Collect feedback and refine the system for better result visualization and report generation.

6. CONCLUSION

CONCLUSION

In conclusion, the development, testing, and implementation of the EduGraph result analysis system require a structured and systematic approach. By following well-defined steps, from backend logic development to frontend implementation, the system is designed to efficiently process and analyse student results.

Through automated result extraction, graphical performance visualization, and PDF report generation, EduGraph eliminates the inefficiencies of manual result processing. Comprehensive testing, including unit, integration, and user acceptance testing (UAT), ensures that the system functions reliably and meets academic performance evaluation requirements.

Deployment in a stable environment, combined with continuous monitoring and maintenance, guarantees system efficiency, accuracy, and usability. Ultimately, the successful implementation of EduGraph provides an automated, accurate, and user-friendly solution for result analysis, offering insightful academic reports and enhancing data-driven decision-making in educational institutions.

.

7. SCOPE FOR FUTURE DEVELOPMENT

SCOPE FOR FUTURE DEVELOPMENT

The EduGraph result analysis system presents significant potential for future enhancements, enabling improved academic performance evaluation and expanding its capabilities to meet evolving institutional needs.

One key area for enhancement is the automation of PDF downloading. By implementing an automated system to fetch student results, EduGraph can further streamline result analysis, eliminating the need for manual file uploads.

Another critical advancement involves historical data comparison. Future iterations of EduGraph can incorporate features that allow institutions to compare student performance across multiple academic years, enabling deeper trend analysis and better decision-making.

Additionally, direct integration with university result portals can be explored. By allowing real-time result extraction from university databases, EduGraph can enhance efficiency and reduce dependency on manually uploaded PDFs.

Expanding data visualization capabilities is another area of future development. By incorporating advanced interactive charts and trend analysis tools, EduGraph can provide more insightful subject-wise and student-wise performance analytics.

Security and usability improvements will also be prioritized. Implementing more robust validation mechanisms for result processing and enhancing the report generation process will ensure better accuracy and reliability.

By adopting continuous improvements, EduGraph aims to evolve into a comprehensive, automated, and insightful result analysis system, facilitating efficient academic evaluation in educational institutions.

8. APPENDIX

APPENDIX

- System source code
- GUI screenshots
- System minimum requirements

8.1 System Source Code

❖ Index Page

home.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1.0, user-scalable=no">
    <title>EduGraph</title>
    <style>
        body {
            margin: 0;
            padding: 0;
            background: linear-gradient(to bottom, #000000 70%, #000033
100%);
            color: white;
            font-family: 'Inter', Arial, Helvetica, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            overflow: hidden;
            position: relative;
        }

        .background-animation {
            position: absolute;
            width: 100%;
            height: 100%;
            top: 0;
            left: 0;
            background: radial-gradient(circle, rgba(0,0,255,0.3) 10%,
rgba(0,0,128,0.2) 50%, rgba(0,0,0,0.9) 100%);
        }

        .ripple {
```

```
            position: absolute;
            width: 100%;
            height: 100%;
            top: 0;
            left: 0;
            pointer-events: none;
        }
        .ripple div {
            position: absolute;
            width: 200px;
            height: 200px;
            border: 2px solid rgba(0, 0, 255, 0.6);
            border-radius: 50%;
            animation: rippleEffect 4s infinite ease-out;
            opacity: 0;
        }
        @keyframes rippleEffect {
            0% {
                transform: scale(0);
                opacity: 1;
            }
            100% {
                transform: scale(3);
                opacity: 0;
            }
        }

        .container {
            width: 80%;
            display: flex;
            justify-content: space-between;
            align-items: center;
            position: relative;
            z-index: 1;
        }
        .left {
            max-width: 50%;
            animation: fadeInLeft 1.5s ease-in-out;
        }
        .left h1 {
            font-size: 4rem;
            font-weight: 400;
            line-height: 1.1;
            margin: 0;
        }
        .left h1 span {
            display: block;
            font-weight: 700;
```

```
            color: #00C2FF;
            animation: colorChange 2s infinite alternate;
        }
        .left p {
            font-size: 1rem;
            font-weight: 400;
            line-height: 1.6;
            margin-top: 20px;
            opacity: 0;
            animation: fadeIn 2s ease-in-out forwards;
            animation-delay: 1s;
        }
        .btn {
            display: inline-block;
            margin-top: 20px;
            padding: 15px 30px;
            background: linear-gradient(90deg, #00d4ff, #000080);
            border: none;
            border-radius: 30px;
            color: white;
            font-size: 18px;
            font-weight: bold;
            text-transform: uppercase;
            cursor: pointer;
            transition: all 0.3s ease-in-out;
            opacity: 0;
            animation: fadeIn 2s ease-in-out forwards;
            animation-delay: 1.5s;
        }
        .btn:hover {
            opacity: 0.85;
            transform: scale(1.05);
            box-shadow: 0 0 20px rgba(0, 0, 255, 0.8);
        }
        .right img {
            max-width: 100%;
            height: 100vh;
            display: block;
            object-fit: cover;
            position: absolute;
            right: 0;
            top: 0;
            animation: slideInRight 1.5s ease-in-out;
        }

        @keyframes fadeInLeft {
            from {
                opacity: 0;
```

```
                transform: translateX(-50px);
            }
            to {
                opacity: 1;
                transform: translateX(0);
            }
        }
        @keyframes slideInRight {
            from {
                opacity: 0;
                transform: translateX(50px);
            }
            to {
                opacity: 1;
                transform: translateX(0);
            }
        }
        @keyframes fadeIn {
            from {
                opacity: 0;
            }
            to {
                opacity: 1;
            }
        }
        @keyframes colorChange {
            from {
                color: #00C2FF;
            }
            to {
                color:rgb(143, 229, 255);
            }
        }

        /* Media Queries for Mobile Devices */
        @media (max-width: 768px) {
            .container {
                width: 90%;
                flex-direction: column;
                align-items: flex-start;
            }
            .left {
                max-width: 100%;
            }
            .left h1 {
                font-size: 3rem;
            }
            .left p {
```

```
                font-size: 0.9rem;
            }
            .btn {
                padding: 12px 24px;
                font-size: 16px;
            }
            .right img {
                height: auto;
                max-width: 100%;
                position: relative;
                margin-top: 20px;
            }
        }

        @media (max-width: 480px) {
            .left h1 {
                font-size: 2.5rem;
            }
            .left p {
                font-size: 0.8rem;
            }
            .btn {
                padding: 10px 20px;
                font-size: 14px;
            }
        }
    </style>
</head>
<body>
    <div class="background-animation"></div>
    <div class="ripple">
        <div style="top: 20%; left: 10%; animation-delay: 0s;"></div>
        <div style="top: 50%; left: 30%; animation-delay: 1s;"></div>
        <div style="top: 70%; left: 60%; animation-delay: 2s;"></div>
        <div style="top: 40%; left: 80%; animation-delay: 3s;"></div>
    </div>

    <div class="container">
        <div class="left">
            <h1>Edu <br><span>Graph</span></h1>
            <p><i>Welcome to EduGraph – Smart Result Analysis! We collect
individual result PDFs and analyze them into a single, insightful report.
Track performance, compare trends, and gain meaningful insights—all in one
place!</i></p>
            <button class="btn" onclick="window.location.href='{% url
'upload' %}'">Get Started</button>
        </div>
        <div class="right">
```

```
            <img src="{% static 'img9.png' %}" alt="EduGraph Image">
        </div>
    </div>
</body>
</html>
```

❖ Uploading Page

upload.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1.0, user-scalable=no">
    <title>Upload PDFs - EduGraph</title>
    <style>
        body {
            background-color: #000000;
            color: white;
            font-family: 'Poppins', sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            margin: 0;
            overflow: hidden;
            position: relative;
        }

        .background-animation {
            position: absolute;
            width: 100%;
            height: 100%;
            top: 0;
            left: 0;
            background: radial-gradient(circle, rgba(0,0,255,0.3) 10%,
rgba(0,0,128,0.2) 50%, rgba(0,0,0,0.9) 100%);
        }

        .ripple {
            position: absolute;
```

```
        width: 100%;
        height: 100%;
        top: 0;
        left: 0;
        pointer-events: none;
    }

    .ripple div {
        position: absolute;
        width: 200px;
        height: 200px;
        border: 2px solid rgba(0, 0, 255, 0.6);
        border-radius: 50%;
        animation: rippleEffect 4s infinite ease-out;
        opacity: 0;
    }

    @keyframes rippleEffect {
        0% {
            transform: scale(0);
            opacity: 1;
        }
        100% {
            transform: scale(3);
            opacity: 0;
        }
    }

    .container {
        background: rgba(0, 0, 0, 0.5);
        padding: 30px;
        border-radius: 20px;
        box-shadow: 0 0 40px rgba(131, 129, 129, 0.8);
        text-align: center;
        width: 90%;
        max-width: 450px;
        position: relative;
        z-index: 1;
        backdrop-filter: blur(10px);
        border: 1px solid rgba(255, 255, 255, 0.1);
        transition: all 0.3s ease-in-out;
        transform: translateY(100vh);
        opacity: 0;
        animation: slideIn 1.2s ease-out forwards;
    }

    @keyframes slideIn {
        0% {
```

```
            transform: translateY(100%);
            opacity: 0;
        }
        100% {
            transform: translateY(0);
            opacity: 1;
        }
    }

    .container:hover {
        transform: scale(1.02);
        box-shadow: 0 0 60px #00d4ff;
    }

    h1 {
        font-size: 32px;
        color: #00d4ff;
        text-transform: uppercase;
        font-weight: bold;
        letter-spacing: 2px;
        margin-bottom: 20px;
    }

    input[type="file"], button {
        width: 100%;
        padding: 15px;
        margin: 15px 0;
        border-radius: 10px;
        border: none;
        font-size: 16px;
        transition: all 0.3s ease-in-out;
        box-sizing: border-box;
    }

    input[type="file"] {
        background: rgba(0, 0, 0, 0.5);
        color: white;
        outline: none;
    }

    input[type="file"]:focus {
        background: rgba(0, 0, 0, 0.4);
        box-shadow: 0 0 10px rgba(0, 0, 255, 0.5);
    }

    button {
        background: linear-gradient(90deg, #00d4ff, #000080);
        color: white;
```

```
            font-size: 18px;
            font-weight: bold;
            cursor: pointer;
        }

        button:hover {
            opacity: 0.85;
            transform: scale(1.05);
            box-shadow: 0 0 20px rgb(124, 118, 118);
        }

        /* Media Queries for Mobile Devices */
        @media (max-width: 768px) {
            .container {
                width: 85%;
                max-width: 400px;
                padding: 25px;
            }

            h1 {
                font-size: 28px;
            }

            input[type="file"], button {
                padding: 12px;
                font-size: 14px;
            }
        }

        @media (max-width: 480px) {
            .container {
                width: 90%;
                max-width: 350px;
                padding: 20px;
            }

            h1 {
                font-size: 24px;
            }

            input[type="file"], button {
                padding: 10px;
                font-size: 12px;
            }
        }
    </style>
</head>
<body>
```

```
    <div class="background-animation"></div>
    <div class="ripple">
        <div style="top: 20%; left: 10%; animation-delay: 0s;"></div>
        <div style="top: 50%; left: 30%; animation-delay: 1s;"></div>
        <div style="top: 70%; left: 60%; animation-delay: 2s;"></div>
        <div style="top: 40%; left: 80%; animation-delay: 3s;"></div>
    </div>

    <div class="container">
        <h1>EduGraph</h1>
        <form method="post" enctype="multipart/form-data"
onsubmit="disableButton(event)">
            {% csrf_token %}
            <input type="file" name="pdf_files" multiple>
            <button type="submit" id="generateInsightsBtn">Generate
Insights</button>
        </form>

        <script>
        function disableButton(event) {
            let button = document.getElementById("generateInsightsBtn");
            button.disabled = true; // Disable button after click
            button.innerText = "Processing..."; // UI feedback
        }
        </script>

    </div>
</body>
</html>
```

❖ Downloading Template

Download.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Edugraph - Downloaded Results</title>
    <style>
        body {
```

```
        font-family: Arial, sans-serif;
        margin: 0;
        padding: 20px;
        font-size: 11px;
        text-align: center;
        width: 100%;
        box-sizing: border-box;
    }
    .date-time {
        text-align: left;
        font-size: 11px;
        margin-bottom: 10px;
    }
    h1, h2 {
        text-transform: uppercase;
        font-size: 20px;
        line-height: 1.5;
        margin: 2px 0;
    }

    h3 {
        margin: 0.5px 0; /* Reduced margin for h3 */
        line-height: 1.5; /* Adjusted line height */
    }
    .student-stats {
        margin: 5px 0; /* Reduced margin for the stats section */
        line-height: 1.5; /* Adjusted line height */

    }
    table {
        width: 100%;
        border-collapse: collapse;
        font-size: 11px;
        margin-top: 10px;
    }
    th, td {
        border: 1px solid black;
        padding: 6px;
        text-align: left;
    }
    th {
        background: #f2f2f2;
        font-weight: bold;
    }
    .center-table th, .center-table td {
        text-align: center;
    }
    .bold {
```

```
            font-weight: bold;
        }

        /* Responsive Adjustments */
        @media (max-width: 600px) {
            body {
                padding: 10px;
            }
            h1 {
                font-size: 20px;
            }
            h2 {
                font-size: 15px;
            }
            table {
                font-size: 10px;
            }
            th, td {
                padding: 4px;
            }
        }
    </style>
</head>
<body>
    <div class="date-time"><strong>Date:</strong> {{ current_date }} |
<strong>Time:</strong> {{ current_time }}</div>
    <h1>🏆 EDUGRAPH RESULT SUMMARY</h1>
    <h2>{{ programme_name }}</h2>
    <div class="student-stats">
        <span class="bold">Total Students:</span> {{ total_students }}
  
        <span class="bold">Total Students Passed:</span> {{ total_pass }}
  
        <span class="bold">Total Students Failed:</span> {{ total_fail }}
    </div>

    <h3>{{ exam_name }}</h3>
    <div class="container">
        <h2>LIST OF FAILED STUDENTS & THEIR SUBJECTS</h2>
        <table>
            <tr>
                <th>Name of Students</th>
                <th>Failed Subjects</th>
            </tr>
            {% for student in failed_students %}
            <tr>
                <td>{{ student.name }}</td>
                <td>
```

```
                    {% for subject in student.failed_subjects.all %}
                        {{ subject.subject_name }}{% if not forloop.last
%},<br>{% endif %}
                    {% endfor %}
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="2">No students failed</td>
            </tr>
            {% endfor %}
        </table>

        <h2>TOP THREE RANK BASED ON SGPA</h2>
        <table class="center-table">
            <tr>
                <th>Rank</th>
                <th>Student's Name & SGPA</th>
            </tr>
            {% for rank, students in top_students %}
            <tr>
                <td>{{ rank }}</td>
                <td>
                    {% for student in students %}
                        {{ student.0 }} - <i>SGPA: {{ student.1 }}</i>{% if
not forloop.last %}<br>{% endif %}
                    {% endfor %}
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="2">No top rankers available</td>
            </tr>
            {% endfor %}
        </table>

        <h2>SUBJECTS & NUMBER OF FAILS</h2>
        <table>
            <tr>
                <th>Subjects</th>
                <th>Number of Fails</th>
            </tr>
            {% for subject, count in subject_fail_count %}
            <tr>
                <td>{{ subject }}</td>
                <td>{{ count }}</td>
            </tr>
            {% empty %}
```

```
            <tr>
                <td colspan="2">No subjects with failed students</td>
            </tr>
            {% endfor %}
        </table>
    </div>
</body>
</html>
```

❖ Analysed Results Page

Results.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Edugraph - Results</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script> <!--
Chart.js Library -->
    <style>
        body {
            background-color: #0a0f1d;
            color: white;
            font-family: 'Poppins', sans-serif;
            margin: 0;
            padding: 0;
            text-align: center;
        }
        .container {
            width: 90%;
            max-width: 1200px;
            margin: 20px auto;
            background: rgba(255, 255, 255, 0.1);
            padding: 20px;
            border-radius: 15px;
            box-shadow: 0 0 15px rgba(0, 212, 255, 0.7);
        }
        h1, h2 {
            color: #00d4ff;
            text-transform: uppercase;
            letter-spacing: 1.5px;
        }
        h1 {
            font-size: 32px;
```

```
            margin-bottom: 20px;
        }
        h2 {
            font-size: 24px;
            margin-top: 30px;
            margin-bottom: 15px;
        }
        p {
            font-size: 18px;
            font-weight: bold;
        }
        .highlight {
            background: rgba(0, 212, 255, 0.2);
            padding: 15px;
            border-radius: 10px;
            margin: 10px auto;
            display: inline-block;
            font-size: 18px;
            font-weight: bold;
            color: #00d4ff;
        }
        table {
            width: 100%;
            margin-top: 15px;
            border-collapse: collapse;
        }
        th, td {
            border: 1px solid #00d4ff;
            padding: 12px;
            text-align: left;
        }
        th {
            background: #00d4ff;
            color: black;
            font-weight: bold;
            text-transform: uppercase;
        }
        tr:hover {
            background: rgba(0, 212, 255, 0.1);
        }
        canvas {
            max-width: 100%;
            margin: 20px auto;
            background: rgba(255, 255, 255, 0.1);
            border-radius: 10px;
            padding: 10px;
        }
        a.download-button {
```

```
            display: inline-block;
            padding: 15px 30px;
            background: linear-gradient(90deg, #00d4ff, #5b6eff);
            color: black;
            font-weight: bold;
            text-decoration: none;
            border-radius: 10px;
            margin: 20px auto;
            transition: all 0.3s ease-in-out;
        }
        a.download-button:hover {
            transform: scale(1.05);
            box-shadow: 0 0 20px rgba(0, 212, 255, 0.8);
        }
        /* Center alignment for Top Three Rank table */
        .center-table {
            text-align: center;
        }
        .center-table th, .center-table td {
            text-align: center;
        }

        /* Media Queries for Mobile Devices */
        @media (max-width: 768px) {
            h1 {
                font-size: 28px;
            }
            h2 {
                font-size: 20px;
            }
            p {
                font-size: 16px;
            }
            .highlight {
                font-size: 16px;
                padding: 10px;
            }
            table {
                font-size: 14px;
            }
            th, td {
                padding: 8px;
            }
            a.download-button {
                padding: 12px 24px;
                font-size: 14px;
            }
        }
```

```
        @media (max-width: 480px) {
            h1 {
                font-size: 24px;
            }
            h2 {
                font-size: 18px;
            }
            p {
                font-size: 14px;
            }
            .highlight {
                font-size: 14px;
                padding: 8px;
            }
            table {
                font-size: 12px;
            }
            th, td {
                padding: 6px;
            }
            a.download-button {
                padding: 10px 20px;
                font-size: 12px;
            }
        }
    </style>
</head>
<body>

    <div class="container">
        <h1>EduGraph Result Summary</h1>
        <h3>{{ programme_name }}</h3>
        <h3>{{ exam_name }}</h3>

        <p class="highlight">Total Students: {{ total_students }}</p>
        <p class="highlight">Total Students Passed: {{ total_pass }}</p>
        <p class="highlight">Total Students Failed: {{ total_fail }}</p>

        <!-- Pie Chart for Pass/Fail -->
        <canvas id="passFailChart"></canvas>

        <h2>List of Failed Students & Their Subjects</h2>
        <table>
            <tr>
                <th>Name of Students</th>
                <th>Failed Subjects</th>
            </tr>
```

```
            {% for student in failed_students %}
            <tr>
                <td>{{ student.name }}</td>
                <td>
                    {% for subject in student.failed_subjects.all %}
                        {{ subject.subject_name }}<br>
                    {% endfor %}
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="2">No students failed</td>
            </tr>
            {% endfor %}
        </table>

        <h2>Top Three Rank based on SGPA</h2>
        <table class="center-table">
            <tr>
                <th>Rank</th>
                <th>Student's Name & SGPA</th>
            </tr>
            {% for rank, students in top_students %}
            <tr>
                <td>{{ rank }}</td>
                <td>
                    {% for student in students %}
                        {{ student.0 }}    -   <i>SGPA: {{
student.1 }} </i><br>
                    {% endfor %}
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="2">No top rankers available</td>
            </tr>
            {% endfor %}
        </table>

        <h2>Subjects & Number of fails</h2>
        <table>
            <tr>
                <th>Subjects</th>
                <th>Number of Fails</th>
            </tr>
            {% for subject, count in subject_fail_count %}
            <tr>
                <td>{{ subject }}</td>
```

```
                <td>{{ count }}</td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="2">No subjects with failed students</td>
            </tr>
            {% endfor %}
        </table>
    </div>

    <!-- Chart.js Script -->
    <script>
        const ctx =
document.getElementById('passFailChart').getContext('2d');
        const totalStudents = {{ total_students }};
        const totalPass = {{ total_pass }};
        const totalFail = {{ total_fail }};

        new Chart(ctx, {
            type: 'pie',
            data: {
                labels: ['Passed', 'Failed'],
                datasets: [{
                    data: [totalPass, totalFail],
                    backgroundColor: ['#00d4ff', '#ff4b4b'], // Blue for
pass, Red for fail
                    hoverOffset: 10
                }]
            },
            options: {
                responsive: true,
                plugins: {
                    legend: {
                        position: 'bottom',
                        labels: {
                            color: 'white'
                        }
                    }
                }
            }
        });
    </script>

    <!-- Download Button -->
    <a href="{% url 'download_results' %}" class="download-button">
        Download Results as PDF
    </a>
```

```
</body>
</html>
```

❖ Views.py

```
import os
import re
import pdfplumber
from collections import defaultdict
from django.shortcuts import render, redirect
from django.conf import settings
from django.core.files.storage import default_storage
from django.template.loader import get_template
from django.http import HttpResponse
from datetime import datetime
from .models import Student, FailedSubject
from xhtml2pdf import pisa

def index(request):
 return render(request, 'home.html')


def extract_exam_details(text):
    lines = text.split("\n")
    exam_name = next((line.replace("Examination", "").strip() for line in
lines if "Examination" in line), "Unknown")
    programme_name = next((match.group(1).strip() for match in
[re.search(r"Programme\s+([^\n]+)", text)] if match), "Unknown")
    return exam_name, programme_name


def check_pass_fail_and_sgpa(pdf_path):
    with pdfplumber.open(pdf_path) as pdf:
        text = "\n".join(filter(None, (page.extract_text() for page in
pdf.pages)))

    exam_name, programme_name = extract_exam_details(text)
    student_name = next((line.replace("Name ", "").strip() for line in
text.split("\n") if line.startswith("Name ")), "Unknown")
    sgpa = float(next((match.group(1) for match in
[re.search(r"SGPA\s*:\s*([\d.]+)", text)] if match), 0.0))
    failed_subjects = [" ".join(line.split()[1:-4]).rsplit(" ", 1)[0].strip()
for line in text.split("\n") if line.endswith("Failed")]

    return "fail" if failed_subjects else "pass", student_name,
failed_subjects, sgpa, exam_name, programme_name
```

```
def process_pdfs(request):
    if request.method == 'POST' and request.FILES.getlist('pdf_files'):
        Student.objects.all().delete()
        FailedSubject.objects.all().delete()

        for pdf_file in request.FILES.getlist('pdf_files'):
            file_path = default_storage.save(pdf_file.name, pdf_file)
            full_path = os.path.join(settings.MEDIA_ROOT, file_path)
            result, student_name, failed_subjects, sgpa, exam_name,
programme_name = check_pass_fail_and_sgpa(full_path)

            student = Student.objects.create(name=student_name, sgpa=sgpa,
result=result)
            FailedSubject.objects.bulk_create([FailedSubject(student=student,
subject_name=subj) for subj in failed_subjects])
            os.remove(full_path)

        request.session.update({'exam_name': exam_name, 'programme_name':
programme_name})
        return redirect('results')
    return render(request, 'upload.html')


def show_results(request):
    students = Student.objects.all()
    failed_students = students.filter(result='fail')

    subject_fail_count = defaultdict(int)
    for subject in FailedSubject.objects.values_list('subject_name',
flat=True):
        subject_fail_count[subject] += 1

    sorted_students = sorted(students, key=lambda s: s.sgpa, reverse=True)

    top_ranks, rank, prev_sgpa = [], 0, None
    for student in sorted_students:
        if prev_sgpa != student.sgpa:
            rank += 1
            if rank > 3:
                break
            top_ranks.append((rank, [(student.name, student.sgpa)]))
        else:
❖               top_ranks[-1][1].append((student.name, student.sgpa))
        prev_sgpa = student.sgpa

    return render(request, 'results.html', {
        'total_students': students.count(),
        'total_pass': students.filter(result='pass').count(),
        'total_fail': failed_students.count(),
```

```
        'failed_students': failed_students,
        'subject_fail_count': list(subject_fail_count.items()),
        'top_students': top_ranks,
        'current_datetime': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        'exam_name': request.session.get('exam_name', 'Unknown'),
        'programme_name': request.session.get('programme_name', 'Unknown')
    })


def generate_pdf(html_content):
    response = HttpResponse(content_type='application/pdf')
    response['Content-Disposition'] = 'attachment; filename="results.pdf"'
    if pisa.CreatePDF(html_content, dest=response).err:
        return HttpResponse("Error generating PDF",
content_type='text/plain')
    return response


def download_results_pdf(request):
    students = Student.objects.all()
    failed_students = students.filter(result='fail')

    subject_fail_count = defaultdict(int)
    for subject in FailedSubject.objects.values_list('subject_name',
flat=True):
        subject_fail_count[subject] += 1

    sorted_students = sorted(students, key=lambda s: s.sgpa, reverse=True)

    top_ranks, rank, prev_sgpa = [], 0, None
    for student in sorted_students:
        if prev_sgpa != student.sgpa:
            rank += 1
            if rank > 3:
                break
            top_ranks.append((rank, [(student.name, student.sgpa)]))
        else:
            top_ranks[-1][1].append((student.name, student.sgpa))
        prev_sgpa = student.sgpa

    context = {
        'total_students': students.count(),
        'total_pass': students.filter(result='pass').count(),
        'total_fail': failed_students.count(),
        'failed_students': failed_students,
        'subject_fail_count': list(subject_fail_count.items()),
        'top_students': top_ranks,
        'current_date': datetime.now().strftime('%d/%m/%y'),
        'current_time': datetime.now().strftime('%I:%M %p'),
```

```
        'exam_name': request.session.get('exam_name', 'Unknown'),
        'programme_name': request.session.get('programme_name', 'Unknown')
    }
    return generate_pdf(get_template('download.html').render(context))
```

❖ urls.py

```
from django.contrib import admin
from django.urls import path
from App import views
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('upload/', views.process_pdfs, name='upload'),
    path('results/', views.show_results, name='results'),
    path('download_results/', views.download_results_pdf,
name='download_results'),  # PDF download route
    path('', views.index, name='home'),

] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

❖ models.py

```
from django.db import models

# Create your models here.
class Student(models.Model):
    name = models.CharField(max_length=255)
    sgpa = models.FloatField()
    result = models.CharField(max_length=10)  # 'pass' or 'fail'

    def __str__(self):
        return self.name

class FailedSubject(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE,
related_name='failed_subjects')
    subject_name = models.CharField(max_length=255)

    def __str__(self):
        return self.subject_name
```

❖ settings.py

```
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.1/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
4(xx$jdfthkzgj0eh(5pd72rmg3otc7&)(b0)(@&bo#p+_@06e'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = ['*']


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'App',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'EduGraph.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
```

```
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'EduGraph.wsgi.application'


# Database
# https://docs.djangoproject.com/en/5.1/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}


# Password validation
# https://docs.djangoproject.com/en/5.1/ref/settings/#auth-password-
validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

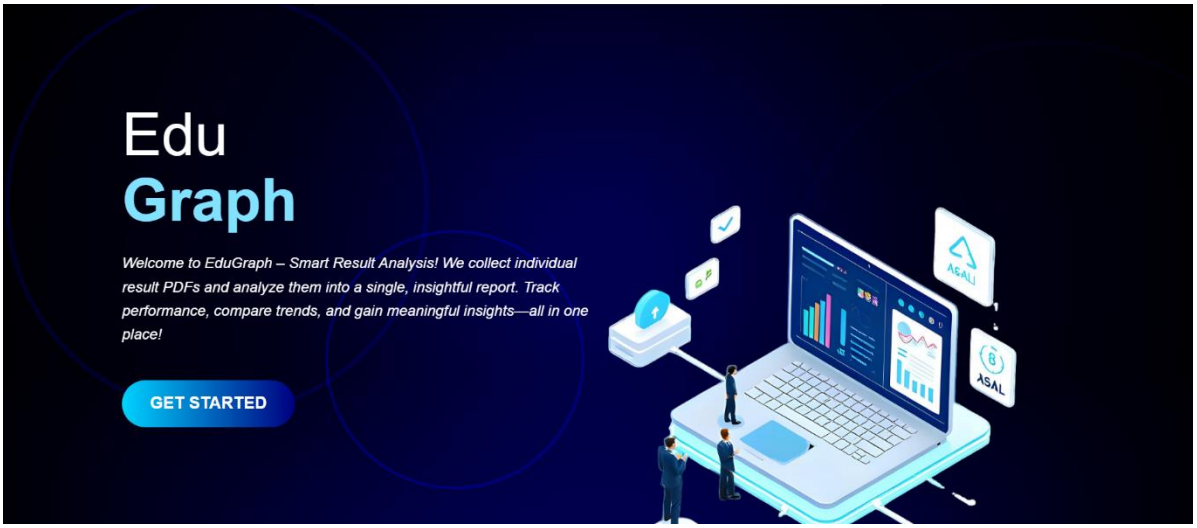
```
# Internationalization
# https://docs.djangoproject.com/en/5.1/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'Asia/Kolkata'

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.1/howto/static-files/

import os


STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'static')



# Add this to settings.py
MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')

# Default primary key field type
# https://docs.djangoproject.com/en/5.1/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
❖
```

❖ requirements.txt

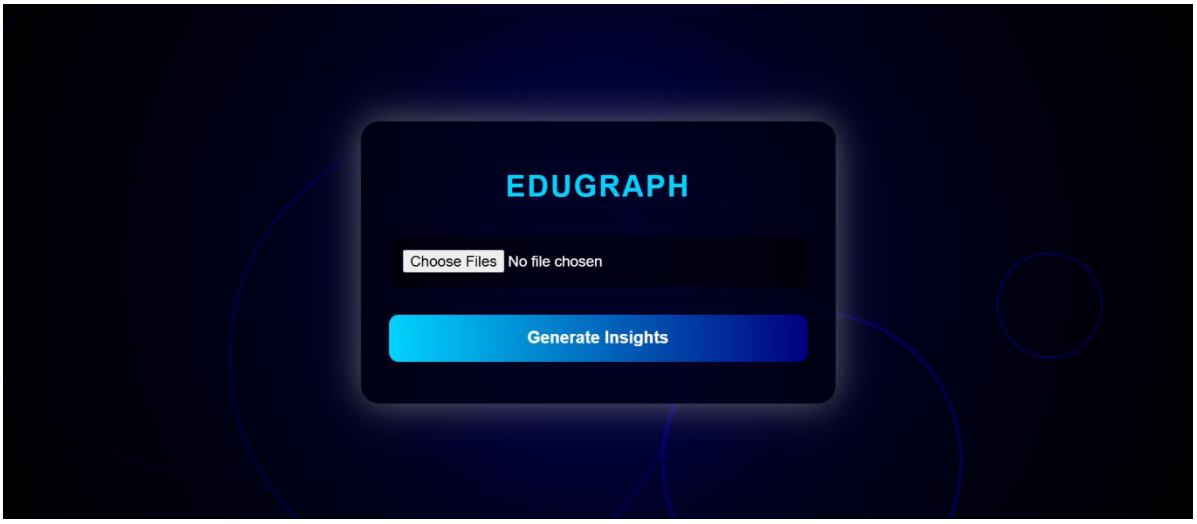
```
Django==5.1.7
pdfplumber==0.11.5
xhtml2pdf==0.2.17
```

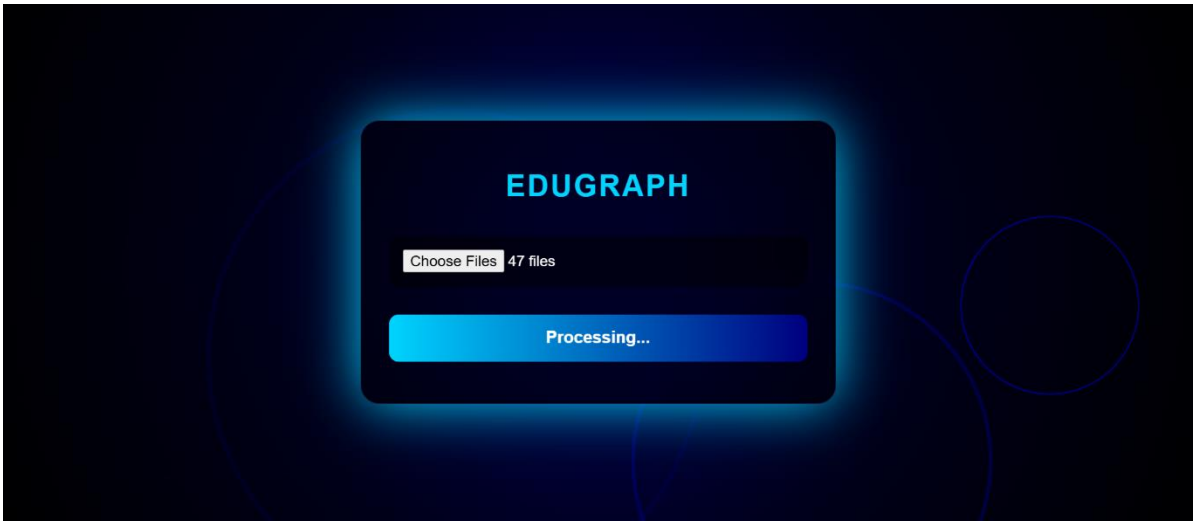
8.2 GUI Screenshots

8.2.1 Home page

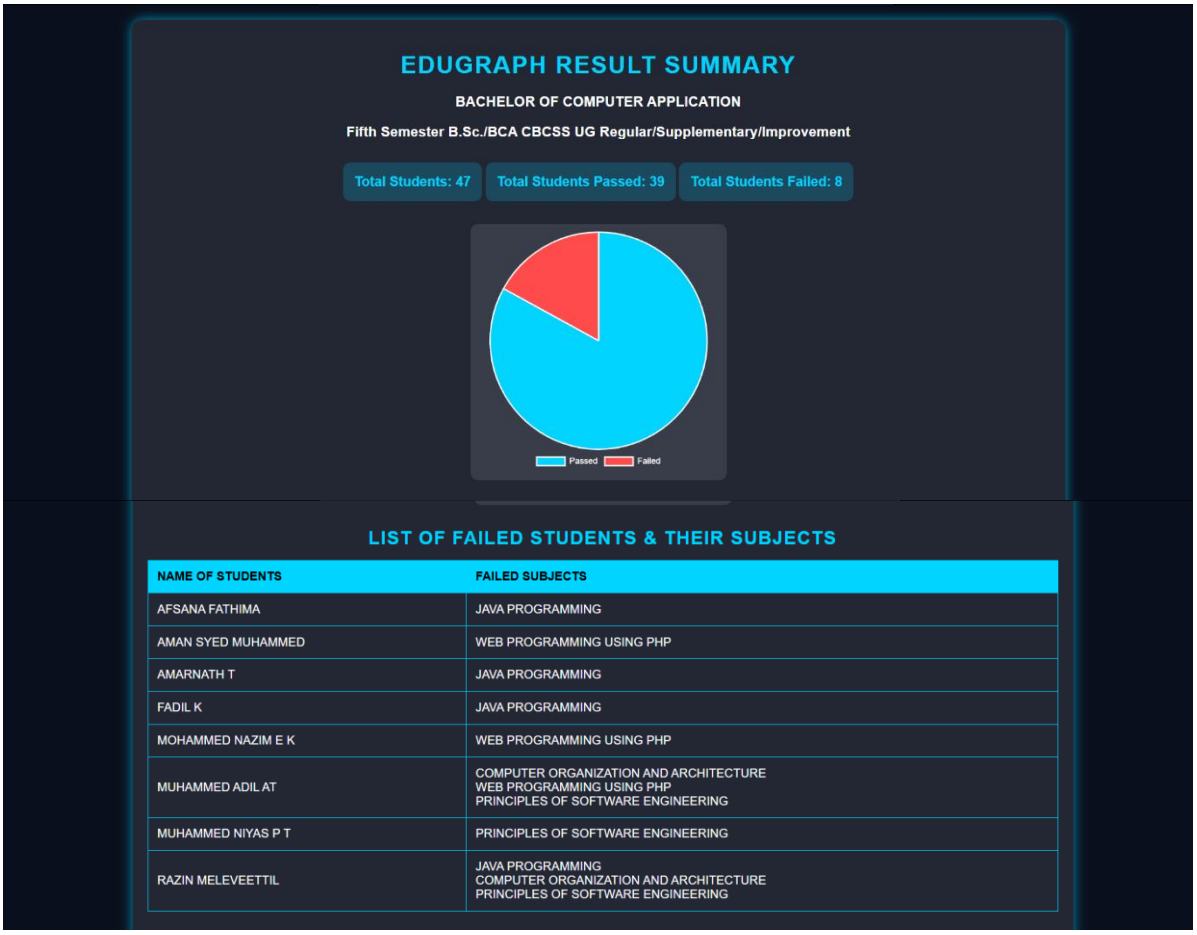


8.2.2 Uploading page

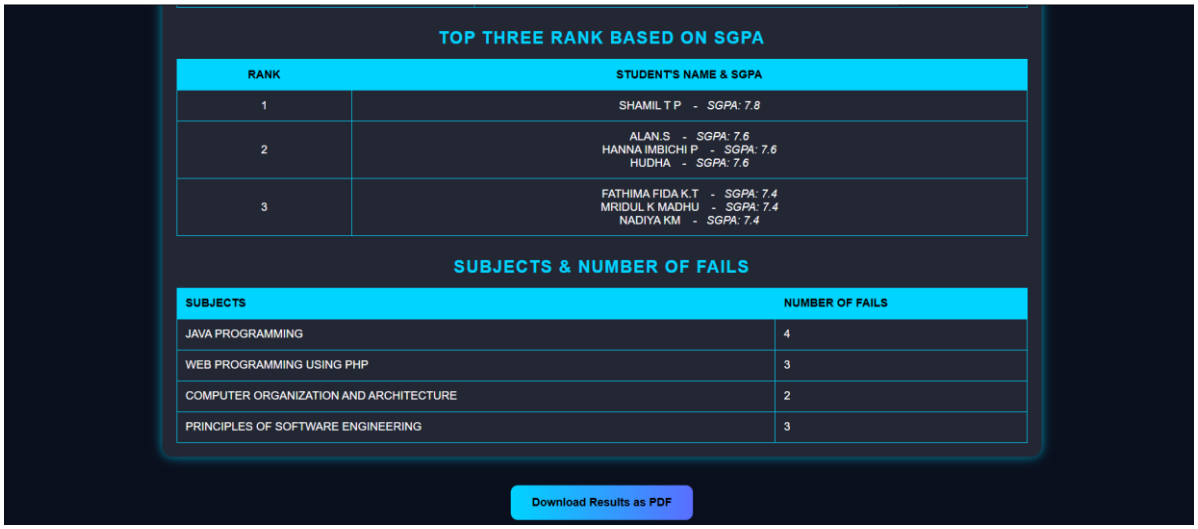




8.2.3 Analysed result page



NAME OF STUDENTS	FAILED SUBJECTS
AFSANA FATHIMA	JAVA PROGRAMMING
AMAN SYED MUHAMMED	WEB PROGRAMMING USING PHP
AMARNATH T	JAVA PROGRAMMING
FADIL K	JAVA PROGRAMMING
MOHAMMED NAZIM E K	WEB PROGRAMMING USING PHP
MUHAMMED ADIL AT	COMPUTER ORGANIZATION AND ARCHITECTURE WEB PROGRAMMING USING PHP PRINCIPLES OF SOFTWARE ENGINEERING
MUHAMMED NIYAS P T	PRINCIPLES OF SOFTWARE ENGINEERING
RAZIN MELEVEETTIL	JAVA PROGRAMMING COMPUTER ORGANIZATION AND ARCHITECTURE PRINCIPLES OF SOFTWARE ENGINEERING



TOP THREE RANK BASED ON SGPA

RANK	STUDENT'S NAME & SGPA
1	SHAMIL T P - *SGPA: 7.8*
2	ALAN.S - *SGPA: 7.6* HANNA IMBICHI P - *SGPA: 7.6* HUDHA - *SGPA: 7.6*
3	FATHIMA FIDA K.T - *SGPA: 7.4* MRIDUL K MADHU - *SGPA: 7.4* NADIYA KM - *SGPA: 7.4*

SUBJECTS & NUMBER OF FAILS

SUBJECTS	NUMBER OF FAILS
JAVA PROGRAMMING	4
WEB PROGRAMMING USING PHP	3
COMPUTER ORGANIZATION AND ARCHITECTURE	2
PRINCIPLES OF SOFTWARE ENGINEERING	3

Download Results as PDF

8.2.4 Downloaded pdf (output)

Date: 14/03/25 | **Time:** 08:11 PM

■ EDUGRAPH RESULT SUMMARY

BACHELOR OF COMPUTER APPLICATION

Total Students: 47 Total Students Passed: 39 Total Students Failed: 8

Fifth Semester B.Sc./BCA CBCSS UG Regular/Supplementary/Improvement

LIST OF FAILED STUDENTS & THEIR SUBJECTS

Name of Students	Failed Subjects
AFSANA FATHIMA	JAVA PROGRAMMING
AMAN SYED MUHAMMED	WEB PROGRAMMING USING PHP
AMARNATH T	JAVA PROGRAMMING
FADIL K	JAVA PROGRAMMING
MOHAMMED NAZIM E K	WEB PROGRAMMING USING PHP
MUHAMMED ADIL AT	COMPUTER ORGANIZATION AND ARCHITECTURE, WEB PROGRAMMING USING PHP, PRINCIPLES OF SOFTWARE ENGINEERING
MUHAMMED NIYAS P T	PRINCIPLES OF SOFTWARE ENGINEERING
RAZIN MELEVEETTIL	JAVA PROGRAMMING, COMPUTER ORGANIZATION AND ARCHITECTURE, PRINCIPLES OF SOFTWARE ENGINEERING

TOP THREE RANK BASED ON SGPA

Rank	Student's Name & SGPA
1	SHAMIL T P - SGPA: 7.8
2	ALAN S - SGPA: 7.6 HANNA IMBICHI P - SGPA: 7.6 HUDHA - SGPA: 7.6
3	FATHIMA RIDA K.T - SGPA: 7.4 MRIDUL K MADHU - SGPA: 7.4 NADIYA KM - SGPA: 7.4

SUBJECTS & NUMBER OF FAILS

Subjects	Number of Fails
JAVA PROGRAMMING	4
WEB PROGRAMMING USING PHP	3
COMPUTER ORGANIZATION AND ARCHITECTURE	2
PRINCIPLES OF SOFTWARE ENGINEERING	3

8.3 System Minimum Requirements

Software Configuration

For the proposed EduGraph result analysis system to function efficiently, the following software components must be installed and running:

- Operating System: Compatible with Windows, Linux, and mobile platforms
- Programming Languages: Python, JavaScript
- Integrated Development Environment (IDE): Visual Studio Code (for development)
- Frontend Technologies: HTML, JavaScript, SCSS
- Backend Technologies: Django
- Other Tools: pdfplumber, xhtml2pdf

Hardware Configuration

EduGraph does not have specific hardware requirements and can be run on any system, including mobile devices. The system is designed to be lightweight and accessible from any device with a web browser.

9. REFERENCES

9.1 Book References:

1. The Definitive Guide to Django:Web Development Done Right , Adrian Holovaty and Jacob kaplan-Moss

2. Python Crash Course , Eric Matthes

3. Django for Beginners , William S.Vincent

9.2 Website References:

1. https://www.djangoproject.com

2. https://www.geeksforgeeks.org/django-tutorial/

3. www.w3schools.com

4. https://chatgpt.com/